Concevez et déployez un système RAG

CONTENU

- Organisez-vous pour...
- Cours - Mettez en pl...
- Ressources pédagog...
- Mission - Créez un a...
- Livrables et soutena...
- Évaluation

SUPPORTS PÉDAGOGIQUES

- Compan... Nouveau
- Cours

Mission - Créez un assistant intelligent pour recommander des événements culturels



Comment allez-vous procéder ?

Cette mission suit un scénario de projet professionnel.
Vous pouvez suivre les étapes pour vous aider à réaliser vos livrables.

Avant de démarrer, nous vous conseillons de :

- lire toute la mission et ses documents liés ;
- prendre des notes sur ce que vous avez compris ;
- consulter les étapes pour vous guider ;
- préparer une liste de questions pour votre première session de mentorat.

Prêt à mener la mission ?

Vous êtes **data scientist freelance**, spécialisé dans le traitement du langage naturel et la création de systèmes intelligents. Vous intervenez en mission pour **Puls-Events**, une entreprise technologique qui développe une plateforme de recommandations culturelles personnalisées.

Puls-Events souhaite tester un nouveau **chatbot intelligent** capable de **répondre à des questions utilisateurs** sur les événements culturels à venir, en s'appuyant sur un système **RAG (Retrieval-Augmented Generation)** combinant recherche vectorielle et génération de réponse en langage naturel.

Votre mission est de livrer un **POC (Proof of Concept)** complet, avec une API exploitable par les équipes produit et marketing. Ce POC devra démontrer la **faisabilité technique, la pertinence métier** et la **performance du système**.



Concevez et déployez un système RAG

CONTENU

- Organisez-vous pour...
- Cours - Mettez en pl...
- Ressources pédagog...
- **Mission - Créez un a...**
- Livrables et soutena...
- Évaluation

SUPPORTS PÉDAGOGIQUES

- Compan... **Nouveau**
- Cours

De : Jérémy

À : moi

Objet : Version fonctionnelle du POC RAG

Salut,

Comme convenu, on a besoin que tu finalises une **version fonctionnelle** du POC pour notre assistant intelligent.

L'objectif est de démontrer aux équipes produit et marketing que notre plateforme peut intégrer un chatbot capable de **répondre aux questions des utilisateurs** en s'appuyant sur les **données d'événements** disponibles via l'API Open Agenda.

Pour ce POC, tu peux cibler une **zone géographique de ton choix**, tant que les événements sélectionnés sont **récents c'est à dire de moins d'un an**.

Voici ce que nous attendons de ta part :

- **Un système RAG fonctionnel** intégrant LangChain, Mistral et Faiss, avec des scripts permettant la reconstruction de l'index vectoriel à partir des données.
- **Une API REST** (FastAPI ou Flask) exposant le système : elle doit permettre d'envoyer une question et de recevoir une réponse augmentée, générée à partir des données vectorisées.
- **Un rapport technique** (PDF ou README) documentant l'architecture, les choix technologiques, les modèles utilisés, les résultats observés et les pistes d'amélioration. Je te rajoute un template en pièce jointe.
- **Un jeu de test annoté** (questions/réponses de référence) pour mesurer la qualité des réponses générées.
- **Des tests unitaires** pour valider l'indexation des données et les performances du système et l'automatisation des métriques d'évaluation.

Tu peux organiser ton dépôt GitHub de manière claire, avec des dossiers pour les scripts, les tests, l'API, et la documentation.

Le but est de pouvoir tester rapidement ta solution.

Nous attendons donc une **démo live** en réunion avec un exemple d'utilisation de l'API. Tu pourras conteneuriser le endpoint dans une image Docker et exécuter cette image en local pour la démonstration que le système est prêt pour un déploiement élargi.

Appuie-toi sur une **présentation PowerPoint** (10 à 15 slides) pour exposer les objectifs, le fonctionnement du système, les résultats, et les perspectives.

Merci d'avance,

Jérémy

Responsable technique Puls-Events

PJ : Template de rapport technique

Cette mission est entièrement guidée.
Suivez les étapes ci-dessous.

Concevez et déployez un système RAG

CONTENU

- Organisez-vous pour...
- Cours - Mettez en pl...
- Ressources pédagog...
- **Mission - Créez un a...**
- Livrables et soutena...
- Évaluation

SUPPORTS PÉDAGOGIQUES

- Compan... **Nouveau**
- Cours

Étapes

Étape 1 - Configurez l'environnement

Avant de développer le système RAG, vous devez mettre en place un environnement de développement reproductible.
Cet environnement vous permettra d'exécuter l'ensemble des composants (scripts de nettoyage, vectorisation, API, etc.) de manière cohérente et stable sur n'importe quelle machine. Vous installerez les bibliothèques nécessaires, préparerez un fichier de dépendances, et testerez le bon fonctionnement des principaux modules.

Prérequis

- Avoir Python ≥ 3.8 installé sur votre machine.
- Connaître les bases de la gestion d'environnement virtuel (venv, conda, ou poetry).
- Avoir une bonne connexion internet pour télécharger les packages nécessaires.

Résultats attendus

- Un environnement de développement (virtualenv ou conda).
- Un Fichier `requirements.txt` ou `environment.yml.`
- Un Fichier `README.md` : objectifs, structure du projet, instructions de reproduction.

Recommandations

Créez un dossier env ou équivalent dans votre dépôt, mais n'ajoutez pas l'environnement au dépôt Git : utilisez un fichier de dépendances pour permettre la reproduction.

- Commencez par tester les imports suivants :

Python

Import faiss

- `from langchain.vectorstores import FAISS`
- `from langchain.embeddings import HuggingFaceEmbeddings`
- `from mistral import MistralClient`
- Ajoutez des commentaires dans votre README.md pour aider un collègue (ou évaluateur) à lancer votre projet sans effort.

Points de vigilance

Concevez et déployez un système RAG

CONTENU

Organisez-vous pour...
Cours - Mettez en pl...
Ressources pédagog...
Mission - Créez un a...
Livrables et soutena...
Évaluation

SUPPORTS PÉDAGOGIQUES

Compan... Nouveau
Cours

- Privilégiez `faiss-cpu` à `faiss-gpu` pour la portabilité.
- Si vous utilisez une API Mistral nécessitant une clé, ne versionnez jamais votre clé d'API : utilisez une variable d'environnement ou un fichier `.env` ignoré par Git.
- Ne pas oublier de tester l'environnement sur une nouvelle machine ou en local après suppression du cache pour simuler une installation "propre".

Étape 2 - Effectuez le pré-processing des données Open Agenda

Description

Récupérer les données d'événements à partir de la plateforme Open Agenda, les filtrer par localisation et période, et structurer ces données pour une utilisation future dans la base de données vectorielle.

Prérequis

- Avoir accès à l'API Open Agenda.
- Avoir compris les paramètres de filtrage souhaités (ville, période, type d'événement).

Résultat attendu

Un jeu de données d'événements propre et structuré, prêt à être indexé. Vous pourrez effectuer des tests unitaires pour vous assurer d'avoir récupérer les données attendues.

Recommandations

- Utiliser l'API Open Agenda pour récupérer les événements.
- Filtrer les événements par localisation et période (1 an d'historique et événements à venir).
- Convertir les descriptions des événements en format vectoriel avec un modèle de NLP comme Mistral.

Points de vigilance

- Attention aux données manquantes ou incorrectes.
- Vérifier la pertinence des filtres pour s'assurer que les événements sont bien ciblés.

Outils

- OpenAgenda API
- Pandas pour la manipulation des données
- Mistral pour la génération de vecteurs

Ressources

- OpenAgenda API Documentation
- Mistral Model Documentation



Concevez et déployez un système RAG

CONTENU

- Organisez-vous pour...
- Cours - Mettez en pl...
- Ressources pédagog...
- **Mission - Créez un a...**
- Livrables et soutena...
- Évaluation

SUPPORTS PÉDAGOGIQUES

- Compan... **Nouveau**
- Cours

Étape 3 - Implémentez la base de données vectorielle avec Faiss

Description

Indexer les descriptions des événements sous forme de vecteurs dans une base de données vectorielle avec Faiss, permettant une recherche rapide basée sur la similarité sémantique.

Prérequis

- Avoir nettoyé les données d'Open Agenda.
- Avoir découpé les textes en chunks avant la vectorisation des textes.
- Vectorisation de chaque chunk et indexation dans la base de données FAISS

Résultat attendu

Une base de données Faiss contenant les événements indexés en fonction de leurs vecteurs sémantiques.

Recommandations

- Utilisez Faiss pour construire un index de recherche sémantique.
- Enregistrez les informations non seulement sur les vecteurs, mais aussi sur les métadonnées des événements (dates, lieux, descriptions).
- Faites des tests de recherche pour vérifier l'efficacité.

Points de vigilance

- S'assurer que l'index est optimisé pour des recherches rapides (utiliser les bons algorithmes dans Faiss).
- Vérifier que tous les événements ont bien été indexés.

Outils

- Faiss pour l'indexation vectorielle
- LangChain pour interagir avec Faiss

Ressource

- Faiss Index Guide

Étape 4 - Intégrez LanChain pour le système RAG

Développer le chatbot intelligent capable de fournir des recommandations personnalisées basées sur les événements indexés, et générer des réponses augmentées à partir des données présentes dans la base de données vectorielle.

Concevez et déployez un système RAG

CONTENU

Organisez-vous pour...
Cours - Mettez en pl...
Ressources pédagog...
Mission - Créez un a...
Livrables et soutena...
Évaluation

SUPPORTS PÉDAGOGIQUES

Compan... Nouveau
Cours

- API Mistral pour générer des réponses naturelles à partir d'un LLM sélectionné sur la plateforme Mistral.

Résultat attendu

Un chatbot capable de fournir des recommandations d'événements et d'interagir avec l'utilisateur.

> Une réponse IA est correcte si elle a le même sens et contient les mêmes informations que la réponse annotée par l'humain. Pour évaluer la qualité des réponses, vous pouvez utiliser des métriques telles que le score de similarité, un score de correspondance exacte (Exact Match), ou une classification manuelle en "correcte / partiellement correcte / incorrecte".

Recommandations

- Utilisez LangChain pour orchestrer les appels entre la base Faiss et le modèle Mistral.
- Configurez des chaînes de traitement pour que le chatbot puisse générer des réponses basées à la fois sur la recherche dans Faiss et sur le modèle de langage.

Points de vigilance

- S'assurer que les réponses générées sont à la fois pertinentes et bien formulées.
- Tester plusieurs scénarios d'interaction pour vérifier la robustesse du chatbot.
- L'historique de conversation entre le bot et l'humain n'est pas nécessaire dans le POC.

Outils

- LangChain pour la gestion des interactions
- Mistral pour la génération de réponses

Ressources

- LangChain Documentation
- Mistral Model

Étape 5 - Créez une API pour exposer le système RAG

Maintenant que votre système RAG est opérationnel, vous devez le rendre **accessible via une API REST**.
L'objectif est que les équipes métier puissent tester la solution en posant une question directement via un **appel HTTP** et recevoir une réponse générée par le système.

Vous utiliserez un micro-framework comme **FastAPI** ou **Flask** pour créer une interface simple permettant de :

- Poser une question en entrée,



Concevez et déployez un système RAG

CONTENU

- Organisez-vous pour...
- Cours - Mettez en pl...
- Ressources pédagog...
- **Mission - Créez un a...**
- Livrables et soutena...
- Évaluation

SUPPORTS PÉDAGOGIQUES

- Compan... **Nouveau**
- Cours

Prérequis

- Savoir manipuler des frameworks d'API Python (FastAPI ou Flask).
- Avoir un système RAG opérationnel sous forme de fonction ou classe.
- Connaître les bases des routes HTTP (GET, POST).

Résultats attendus

- Une API REST locale exposant le système RAG
- Un endpoint `/ask` (POST) qui prend une question et renvoie une réponse générée
- Un endpoint `/rebuild` (GET ou POST) pour reconstruire la base vectorielle à la demande
- Une documentation Swagger générée automatiquement (si FastAPI est utilisé)
- Un test fonctionnel de l'API dans un script ou fichier `api_test.py`

Recommandations

- Séparez bien votre logique métier (RAG) de votre code d'API.
- Documentez chaque route (ce qu'elle fait, quelles données elle attend).
- Si vous utilisez FastAPI, tirez parti de la documentation interactive générée automatiquement à l'adresse `/docs` .
- Pensez à la réutilisabilité : votre système peut être encapsulé dans une classe ou une fonction centrale importée dans votre API.

> Pour évaluer automatiquement la qualité des réponses générées, utilisez une bibliothèque comme **Ragas**, qui permet de comparer les réponses de votre système à des réponses humaines annotées.Elle fournit des métriques précises pour la pertinence, la fidélité au contexte, et la couverture documentaire. Cela facilite l'intégration dans un **pipeline de test automatisé** via script ou CI (par exemple GitHub Actions). **Ragas** peut parfaitement être utilisé dans un **script Python de test automatique (evaluate_rag.py)** lancé dans une étape GitHub Actions.

Points de vigilance

- Vérifiez que votre API gère correctement les erreurs (questions vides, mauvaise requête...).
- Ne laissez pas d'informations sensibles exposées (comme une clé d'API).
- Faites attention aux performances : évitez de relancer toute la chaîne à chaque appel si ce n'est pas nécessaire.
- Protégez les endpoints sensibles (comme `/rebuild`) s'ils étaient un jour exposés publiquement.

Outils

- FastAPI ou Flask

Concevez et déployez un système RAG

CONTENU

Organisez-vous pour...
Cours - Mettez en pl...
Ressources pédagog...
Mission - Créez un a...
Livrables et soutena...
Évaluation

SUPPORTS PÉDAGOGIQUES

Compan... Nouveau
Cours

- Python requests ou httpx pour les tests d'API
- Ragas.

Ressources

- FastAPI – Quickstart
- Swagger UI via FastAPI
- Flask – Documentation
- Exemple d'API RAG avec LangChain
- Chapitre 4, partie 2 du cours Mettez en place un RAG pour un LLM Évaluez et améliorez votre système RAG

Étape 6 - Conteneurisez, déployez localement et préparez votre démonstration

Il est temps de **préparer la livraison finale de votre projet** en rendant votre système exécutable localement via un conteneur Docker.

Dans cette étape, vous rendrez l'ensemble de votre système exécutable localement et présentable lors de la soutenance.

Vous présenterez votre système RAG fonctionnel à l'aide d'un **endpoint API exposé dans le conteneur**, accompagné d'une **démonstration live** et d'une **présentation claire et structurée** pour les parties prenantes métier.

Prérequis

- Tous vos scripts (prétraitement, vectorisation, API) doivent être fonctionnels
- Votre environnement doit être bien configuré (`README` , `requirements.txt` , etc.)
- `Dockerfile` prêt à générer une image exécutable.
- Vous devez avoir testé le système RAG de bout en bout.

Résultats attendus

- Un script de build de l'index vectoriel et un fichier docker permettant de builder et de run l'api en local
- Une démo live fluide : question posée → réponse retournée
- Une présentation PowerPoint (10 à 15 slides) incluant :
  - Objectif et contexte du projet
  - Architecture technique (incluant la conteneurisation et l'API)
  - Données utilisées et modèles choisis
  - Résultats et évaluation du système
  - Perspectives d'amélioration
- Une capacité à répondre à des questions techniques et métier lors de la soutenanc

Concevez et déployez un système RAG

CONTENU

- Organisez-vous pour...
- Cours - Mettez en pl...
- Ressources pédagog...
- **Mission - Créez un a...**
- Livrables et soutena...
- Évaluation

SUPPORTS PÉDAGOGIQUES

- Compan... **Nouveau**
- Cours

- Testez l'exécution complète.
- Préparez 2-3 scénarios d'usage réalistes pour la démo (par ex. : "Quels événements jazz à Paris cette semaine ?").
- Structurez bien votre présentation :
  - commencez par le problème,
  - puis la solution,
  - les résultats
  - et enfin les perspectives.
- Soyez prêt à **justifier vos choix** techniques, et à parler à la fois aux profils techniques et non techniques.

Points de vigilance

- Évitez de dépendre d'une connexion Internet instable pour la démo (préparez une version locale).
- Vérifiez que toutes les dépendances sont bien installées et compatibles.
- Relisez votre présentation et vérifiez qu'elle est compréhensible sans jargon inutile.
- Préparez une courte explication métier de ce qu'est un système RAG.

Outils recommandés

- Docker, Docker Compose (facultatif)
- FastAPI (recommandé pour Swagger UI) ou Flask
- Terminal, Postman ou navigateur pour tester les endpoints
- PowerPoint, Google Slides ou Pitch pour la présentation
- GitHub pour versionner votre projet

Vérifiez votre travail et faites le point avec votre mentor

Avez-vous une suggestion pour nous ?

Précédent Suivant